COMPUTING 2

Parallel Computing Midterm Study Guide

Complete lecture review + solved practice midterm + derivations + worked data

Coverage	Performance models, CMP/vector, GPU architecture, CUDA, OpenMP, and MPI fundamentals
Source	GitHub lecture snapshot: de15932967037bdfb0103ddacc07721fe17c9849
Prepared	2026-10-16
Goal	One document for concepts, calculations, code analysis, and final review

Recommended sequence
Read Chapters 1 and 4 first. Solve Chapter 7 without a time limit, then solve Chapter 8 under time pressure. Use Chapter 9 for the final repetition cycle.

Table of Contents

Table of Contents 2

0. Scope and Exam Map 4

Lecture Material Map . . . 4

1. Parallel Performance Models 5

1.1 Metrics and overhead . . . 5

1.2 Amdahl's law: fixed-size speedup . . . 5

1.3 Gustafson's law: fixed-time scaled speedup . . . 6

1.4 Roofline model . . . 6

2. Parallel Algorithm Design and Communication 8

2.1 Decomposition, dependencies, and granularity . . . 8

2.2 Scheduling and load balance . . . 8

2.3 Communication and collectives . . . 8

3. CMP, Superscalar, SMT, and Vector Execution 9

3.1 Layers of parallelism . . . 9

3.2 Superscalar and out-of-order execution . . . 9

3.3 Vector/SIMD . . . 9

3.4 SMT vs. multicore vs. SIMD . . . 9

3.5 Coherence, consistency, and false sharing . . . 10

4. GPU Architecture: SIMT, Warps, and Memory 11

4.1 CPU and GPU design goals . . . 11

4.2 CUDA execution hierarchy . . . 11

4.3 NVIDIA, AMD GCN, and Arm Valhall . . . 11

4.4 Control divergence . . . 11

4.5 GPU memory hierarchy . . . 12

4.6 Coalescing . . . 12

4.7 Shared-memory banks and registers . . . 12

5. CUDA Programming: Indexing, Synchronization, and Tiling 13

5.1 Host-device flow . . . 13

5.2 One- and two-dimensional indexing . . . 13

5.3 Barrier and transpose race . . . 13

5.4 Matrix-multiplication load count . . . 14

5.5 Tiled GEMM skeleton . . . 14

5.6 Resources and occupancy . . . 14

6. Pthreads, OpenMP, MPI, and Hybrid Models 16

6.1 Model comparison . . . 16
6.2 OpenMP fork-join and worksharing . . . 16
6.3 Synchronization choices . . . 17
6.4 OpenMP memory model and tasks . . . 17
6.5 MPI matrix-vector example . . . 17

7. Solved Practice Midterm 18

7.1 Q1 True/False . . . 18
7.2 Q2 Terms . . . 18
7.3 Q3 Amdahl calculation . . . 18
7.4 Q4 Two-section index . . . 18
7.5 Q5 Transpose correctness . . . 19
7.6 Q6 Coalesced first elements . . . 19
7.7 Q7 Matrix-multiplication requests . . . 19
7.8 Q8 Shared-variable instances . . . 19
7.9 Q9 Why shared memory instead of registers? . . . 19

8. Additional Exam-Style Questions 20

Five-step CUDA code check . . . 21

9. Final Cram Sheets 22

9.1 Formulas to know . . . 22
9.2 Instant true/false . . . 22
9.3 Memory-choice checklist . . . 22
9.4 Last self-check . . . 23

10. References and Further Reading 24

Architecture-dependent values . . . 24

0. Scope and Exam Map

The lecture set moves from parallel performance models to CPU and GPU architecture, CUDA programming, and shared/distributed programming interfaces. The practice midterm concentrates on three abilities: **computing performance limits**, **reasoning about GPU execution and memory**, and **judging CUDA code for correctness and access efficiency**.

Priority	You must be able to	Practice linkage
A	Compute Amdahl limits; classify Roofline bottlenecks; derive CUDA indices; find missing synchronization; count tiled loads	Q1, Q3-Q9
B	Explain warp/wavefront, divergence, vector execution, superscalar issue, SMT, GPU memory spaces, and Valhall scheduling	Q1-Q2
C	Use OpenMP data clauses and reduction; distinguish MPI collectives; reason about false sharing and task decomposition	Q2 and lecture coverage

Lecture Material Map

Lecture files	Core topics represented here
Intro to Parallel Computing 1-3	Motivation, performance metrics, decomposition, scheduling, programming models, pthreads, OpenMP, MPI, and CUDA
CMP and Vector Computers	Multicore processors, ILP, superscalar issue, SMT, SIMD/vector execution, and memory limits
GPU Architectures 1-2	Throughput cores, SIMT, warp/wavefront execution, NVIDIA/AMD/Arm designs, divergence, and latency hiding
CUDA basics_extended	Grid/block/thread mapping, memory spaces, indexing, coalescing, shared-memory tiling, and synchronization
intro_to_HPC_OpenMP / omp-hands-on	Fork-join, worksharing, data environment, reduction, scheduling, false sharing, and tasks
Computing2 practice midterm	True/false reasoning, terminology, calculations, code correctness, and memory-request counting

How to write exam answers

For true/false items, add one causal sentence. For calculations, state the assumption and formula before substitution. For code questions, check bounds, ownership, synchronization, access pattern, and resource limits in that order.

1. Parallel Performance Models

1.1 Metrics and overhead

Let T1 be serial execution time and Tp be execution time on p processing elements. Speedup measures improvement, efficiency measures utilization per processing element, and cost measures aggregate processor-time.

$$S(p) = T1 / Tp \quad E(p) = S(p) / p \quad C(p) = p\,Tp$$

Effect	Cause	Observed behavior	Typical response
Serial bottleneck	Nonparallel fraction s	Speedup saturates as p grows	Improve or restructure the serial path
Communication/synchronization	Barrier, lock, or messages	Overhead grows with scale	Aggregate messages, overlap work, improve locality
Load imbalance	Unequal work or boundary cases	The slowest worker sets completion time	Dynamic scheduling or finer tasks
Tasks too fine	Scheduling cost dominates	More threads make execution slower	Coarsen tasks or chunks
Memory bottleneck	Bandwidth, latency, irregular access	More cores do not raise FLOP/s	Reuse, tiling, and coalescing

1.2 Amdahl's law: fixed-size speedup

Normalize T1 to 1. Let s be the serial fraction and 1-s the ideally parallel fraction. On p processors, the serial part still costs s while the parallel part costs (1-s)/p.

$$Tp = s + (1-s)/p \text{ therefore } S_A(p) = 1 / [s + (1-s)/p]$$

As p approaches infinity, the parallel term approaches zero but s remains. The maximum possible speedup is therefore 1/s. A small serial fraction becomes a hard system-level ceiling.

$$\lim(p \to \infty)\ S_A(p) = 1/s$$

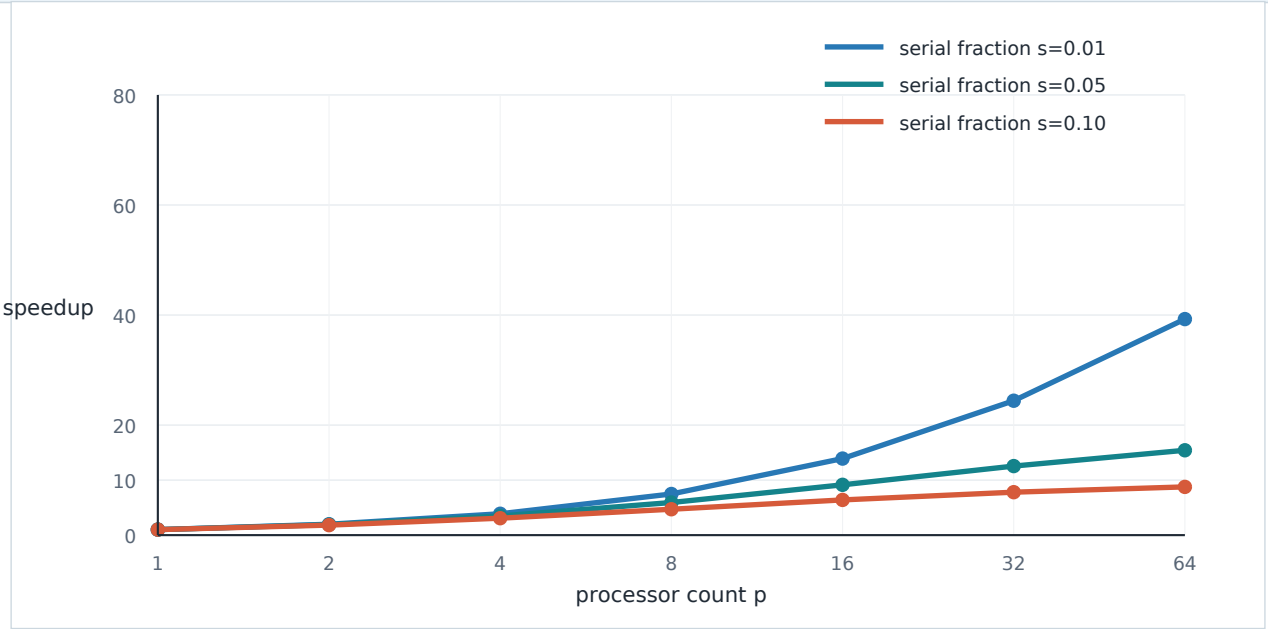


Figure 1. Amdahl speedup for three serial fractions. A 10% serial fraction limits speedup to 10x.

p	s=1%	s=5%	s=10%
1	1.00	1.00	1.00
2	1.98	1.90	1.82
4	3.88	3.48	3.08
8	7.48	5.93	4.71
16	13.91	9.14	6.40
32	24.43	12.55	7.80
64	39.26	15.42	8.77

Practice Q3
With s=0.10 and p=8: S=1/(0.10+0.90/8)=4.7059. With infinitely many processors: Smax=1/0.10=10.

1.3 Gustafson's law: fixed-time scaled speedup

Amdahl fixes problem size. Gustafson instead asks how much larger a problem p processors can solve in the same elapsed time. If parallel execution time is normalized to 1, with serial time s and parallel time 1-s, a single processor would spend p(1-s) on the scalable part.

$$S_G(p) = s + p(1-s) = p - s(p-1)$$

Law	What is fixed?	Question answered	Caution
Amdahl	Problem size	How much faster is the same problem on p processors?	The serial fraction sets a strong ceiling
Gustafson	Elapsed time	How much larger a problem can p processors solve?	The parallel workload must scale

Strong vs. weak scaling
Strong scaling keeps the problem fixed and is closely related to Amdahl. Weak scaling increases problem size with processor count and asks whether time remains approximately constant when work per processor is fixed.

1.4 Roofline model

Let a kernel perform W floating-point operations while moving Q bytes to or from DRAM. Operational intensity is I=W/Q [FLOP/byte]. If memory bandwidth is B [byte/s] and peak compute performance is Ppeak [FLOP/s], the kernel is bounded by both roofs.

$$P_{attainable} <= min(P_{peak}, B \times I) \quad I_{ridge} = P_{peak} / B$$

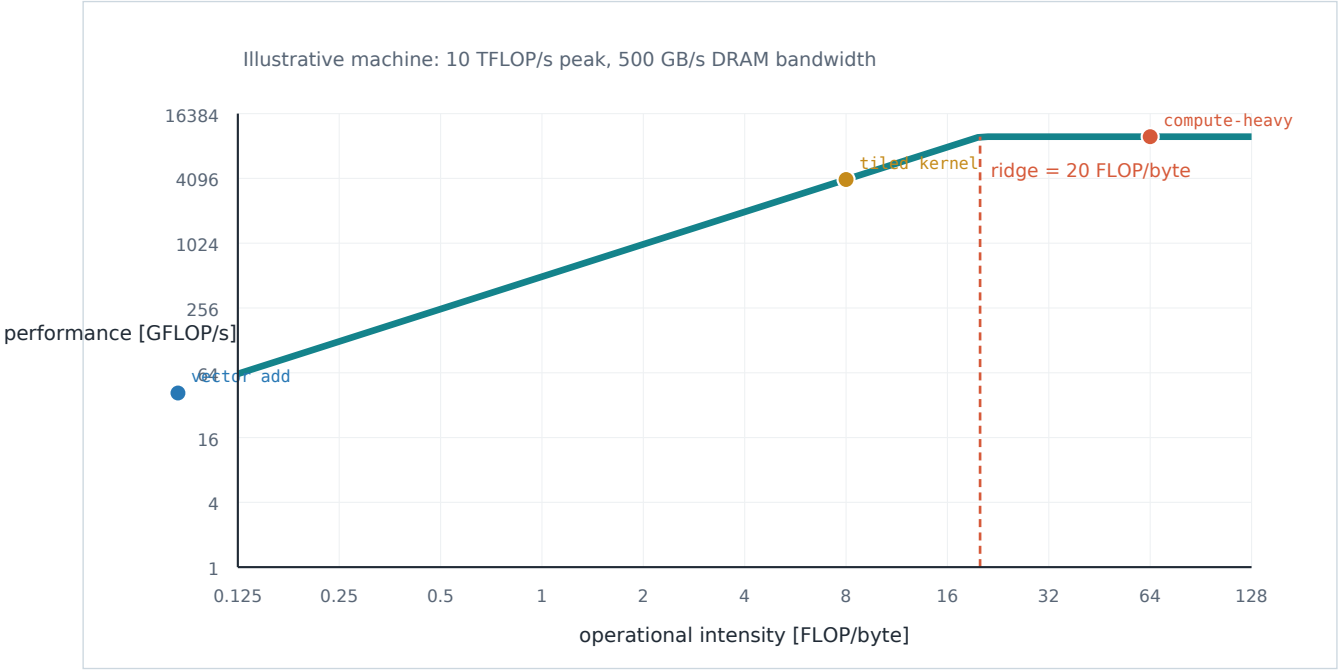


Figure 2. Illustrative Roofline. The sloped region is memory-bound; the horizontal region is compute-bound.

If I is below the ridge point, prioritize reuse, tiling, coalescing, and reduced data traffic. Above the ridge point, prioritize vectorization, instruction mix, specialized compute units, and execution efficiency. Increasing I does not raise performance after the compute roof is reached, so Practice Q1(f) is false.

Kernel	Work / traffic	I	Memory roof at 500 GB/s
C=A+B, float	1 FLOP, 12 bytes	0.083 FLOP/B	about 41.7 GFLOP/s
SAXPY y=ax+y	2 FLOP, 12 bytes	0.167 FLOP/B	about 83.3 GFLOP/s
Naive GEMM	Reuse depends on cache	low to moderate	implementation-dependent
Tiled GEMM	Loaded data reused within a tile	grows with T	moves toward compute roof

2. Parallel Algorithm Design and Communication

2.1 Decomposition, dependencies, and granularity

Parallelization is not merely creating more threads. It exposes independent work while preserving dependencies. Begin with a measured serial baseline, locate hotspots, build the task/data dependency picture, and then balance parallelism against scheduling overhead and locality.

- Task decomposition splits different computations into tasks, such as recursive subproblems or graph-frontier work.
- Data decomposition partitions rows, blocks, tiles, or domains and applies similar computation to each partition.
- Pipeline decomposition overlaps stages but requires correct stage dependencies and buffering.
- The critical path is the longest dependent path and limits execution time even with unlimited processors.

For total work W and critical-path length D: Tp >= max(W/p, D). Available parallelism is approximately W/D.

2.2 Scheduling and load balance

Policy	Strength	Weakness	Best fit
static	Low overhead and predictable locality	Poor under variable work	Uniform iteration costs
dynamic	Idle workers request more chunks	Runtime and synchronization overhead	Irregular iteration costs
guided	Large chunks first, smaller chunks later	Runtime-dependent behavior	Balance overhead against tail imbalance
work stealing	Good for irregular recursive tasks	Deque and migration overhead	Nonstrict task graphs

2.3 Communication and collectives

A common message-cost model is Tcomm(n)=alpha+beta n. Alpha is startup latency; beta is time per byte. Small messages are latency-dominated, while large messages are bandwidth-dominated. Combine small messages, overlap communication with computation, and maximize data locality.

Collective	Input -> output	Meaning
broadcast	One root value -> every rank	Distribute configuration or a shared vector
gather	One piece per rank -> root	Collect distributed results at one rank
all-gather	One piece per rank -> complete concatenation at every rank	Every process receives every piece
reduce	One value per rank -> combined value at root	Apply sum, max, or another associative operation
all-reduce	One value per rank -> combined value at every rank	Typical gradient aggregation pattern
barrier	All participants arrive before any pass	Collective synchronization, not necessarily data movement

Why a barrier is collective
Its meaning depends on the whole participant set. No participant may pass until every participant has reached the synchronization point.

3. CMP, Superscalar, SMT, and Vector Execution

3.1 Layers of parallelism

Layer	Unit	Mechanisms	Goal
ILP	Instructions in one thread	Pipelining, superscalar issue, out-of-order execution	Execute independent instructions together
DLP	Data elements	SIMD and vector instructions	Apply one operation to many elements
TLP	Threads	Multicore/CMP and SMT	Run independent threads or hide latency
Process level	Independent address spaces	MPI and distributed memory	Scale across nodes

3.2 Superscalar and out-of-order execution

A superscalar processor can fetch, issue, and execute more than one instruction per cycle using multiple functional units. Common dynamic designs inspect dependencies and resource readiness, then issue ready instructions. Strictly speaking, superscalar does not logically require out-of-order execution; in-order superscalar implementations are possible.

Practice Q1(k) nuance
In the lecture's dynamic-superscalar context, the expected answer is true: hardware dynamically selects ready instructions for multiple units. A precise answer can state that superscalar and out-of-order execution are related but not identical concepts.

3.3 Vector/SIMD

A vector instruction applies the same operation across multiple lanes. Replacing many scalar instructions with one vector instruction reduces instruction fetch/decode bandwidth and loop overhead. Actual speedup is limited by alignment, memory bandwidth, dependencies, and tail handling.

3.4 SMT vs. multicore vs. SIMD

Mechanism	Replicated state/resources	Shared resources	Main tradeoff
Multicore	Core pipelines and major execution resources	Some last-level cache and memory system	True core-level throughput at area/power cost
SMT	Architectural state for multiple hardware threads	One core's issue, execution, and cache resources	Fills empty slots but introduces contention
SIMD	Lane datapaths	One control/instruction stream	Efficient regular DLP; weak under irregular control

3.5 Coherence, consistency, and false sharing

Coherence concerns how cached copies of the same memory location agree. Consistency concerns the order in which memory operations to possibly different locations become visible. Synchronization establishes the ordering required by the program.

False sharing occurs when threads update different variables that reside in the same cache line. Correctness may be unaffected, yet repeated coherence invalidations destroy performance. Padding per-thread accumulators or assigning contiguous blocks can help.

4. GPU Architecture: SIMT, Warps, and Memory

4.1 CPU and GPU design goals

CPU	GPU
Low latency and strong single-thread performance	High throughput with many concurrent threads
Large caches and sophisticated branch/out-of-order machinery	More chip area devoted to arithmetic throughput and thread state
A few threads should progress quickly	Many warps alternate to hide latency
Strong on irregular control	Strong on regular data-parallel work

Calling GPU cores 'simpler' means that more area and power are allocated to throughput rather than to aggressive control logic and large per-core caches.

4.2 CUDA execution hierarchy

Software unit	Hardware view	Key constraint
grid	All blocks in a kernel launch	No ordinary block-to-block barrier inside a kernel
thread block	Cooperative unit resident on one SM	Scope of shared memory and __syncthreads()
warp	NVIDIA's basic SIMT execution group	32 threads; architecture-defined
thread	Per-lane state and data index	Participates in the warp instruction stream

All warps of a block reside on the same SM. An SM can hold warps from several blocks. When one warp waits for memory, the scheduler can issue a ready warp. Sufficient active warps help hide latency, but maximum occupancy is not automatically maximum performance.

4.3 NVIDIA, AMD GCN, and Arm Valhall

Family	Execution group	Typical width	Scheduling point
NVIDIA CUDA	warp	32 threads	An SM warp scheduler issues ready warps
AMD GCN	wavefront	64 work-items	Wavefronts of a work-group reside on one CU
Arm Valhall	warp	implementation-dependent	Hardware dynamic instruction scheduling and superscalar issue

Exam trap
Software chooses block/work-group dimensions, but the architecture chooses warp/wavefront width. Therefore Practice Q1(m) is false.

4.4 Control divergence

Divergence occurs when threads in one warp take different branch paths. The hardware changes active masks, executes the paths in separate steps, and reconverges. This preserves correctness but reduces the fraction of useful lanes. If all threads select the same path, there is no divergence.

```
if (threadIdx.x % 2 == 0)  // half of a warp is true
    y = fast_path();
```

```
else
    y = slow_path();              // paths execute with different masks
```

4.5 GPU memory hierarchy

Space	Scope/lifetime	Management	Exam point
register	Thread private / thread lifetime	Compiler and hardware	Fast, not shareable; pressure can lower occupancy or spill
shared memory / scratchpad	Block shared / block lifetime	Explicit program loads/stores	Staging, reuse, barriers, bank conflicts
L1/L2 cache	SM/device levels	Hardware	Automatic reuse depends on locality
constant/texture	Read-only access paths	Specialized caches	Broadcast or 2D spatial locality
global memory	Device allocation lifetime	Host allocation plus kernel access	Large, high latency, coalescing matters
local memory	Logically thread-private; physically device memory	Compiler spills/arrays	The name 'local' does not imply low latency
Unified Memory	CPU/GPU unified addressability	cudaMallocManaged and migration	Convenient, but page migration/prefetch matter

Scratchpad semantics
A cache fills automatically, but shared scratchpad data movement is explicit. Code loads global data into shared memory and must synchronize cross-thread producer-consumer phases when required.

4.6 Coalescing

Coalescing combines addresses requested by one warp memory instruction into as few memory transactions as possible. Adjacent threads accessing adjacent 4-byte elements is the standard efficient pattern. The practical metric is useful bytes divided by transferred bytes.

```
// Good: lane t reads base+t
int i = blockIdx.x * blockDim.x + threadIdx.x;
x = A[i];

// Potentially poor: lane t reads base+stride*t
x = A[base + stride * threadIdx.x];
```

4.7 Shared-memory banks and registers

Shared memory is divided into banks. If threads in a warp access distinct addresses in the same bank, those accesses can serialize. A same-address read can be broadcast. Padding a transpose tile from tile[T][T] to tile[T][T+1] is a common way to change bank mapping.

Question	Registers	Shared memory
Who can access it?	Only the owning thread	Every thread in the block
Synchronization?	None for thread-private use	Needed for cross-thread production/consumption
Primary value	Private temporaries and high bandwidth	Cooperative loads, sharing, reuse, access reordering
Primary risk	Register pressure, occupancy loss, spills	Capacity, barriers, bank conflicts

5. CUDA Programming: Indexing, Synchronization, and Tiling

5.1 Host-device flow

```
float *dA, *dB, *dC;
cudaMalloc(&dA, n*sizeof(float));
cudaMalloc(&dB, n*sizeof(float));
cudaMalloc(&dC, n*sizeof(float));
cudaMemcpy(dA, hA, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(dB, hB, bytes, cudaMemcpyHostToDevice);

int block = 256;
int grid = (n + block - 1) / block;
vecAdd<<<grid, block>>>(dA, dB, dC, n);
cudaGetLastError();
cudaDeviceSynchronize();

cudaMemcpy(hC, dC, bytes, cudaMemcpyDeviceToHost);
cudaFree(dA); cudaFree(dB); cudaFree(dC);
```

A kernel launch may be asynchronous with respect to the host. Check launch errors separately from execution errors. cudaMallocManaged() creates a Unified Memory allocation.

5.2 One- and two-dimensional indexing

A one-dimensional global index is the number of threads in earlier blocks plus the thread's local offset.

i = blockIdx.x x blockDim.x + threadIdx.x

If each block covers two consecutive sections of B=blockDim.x elements and every thread first handles one element in section 1, use i=blockIdx.x*(2B)+threadIdx.x. The second element is i+B. Both warp loads are contiguous.

```
int i = blockIdx.x * (2 * blockDim.x) + threadIdx.x;
if (i < n) C[i] = A[i] + B[i];
i += blockDim.x;
if (i < n) C[i] = A[i] + B[i];
```

Assigning consecutive elements per thread with i=2*(blockIdx.x*blockDim.x+threadIdx.x) is functionally plausible, but the warp's first access is 0,2,4,... and touches more memory segments. Per-thread contiguity is not the same as warp-level coalescing.

For row-major 2D data: col=blockIdx.x*blockDim.x+threadIdx.x, row=blockIdx.y*blockDim.y+threadIdx.y, and idx=row*width+col.

5.3 Barrier and transpose race

Each thread first writes one global element into a shared tile, then reads an element written by another thread. The store and cross-thread load phases require block-wide synchronization.

```
__shared__ float tile[T][T];
int x = blockIdx.x * T + threadIdx.x;
int y = blockIdx.y * T + threadIdx.y;
int idx = y * width + x;

tile[threadIdx.y][threadIdx.x] = A[idx];
__syncthreads();
A[idx] = tile[threadIdx.x][threadIdx.y];
```

Without a barrier, warp 0 may begin transposed loads while the required values from warp 1 have not yet been stored. A one-warp block may appear to work because the warp executes the store instruction before the load instruction, but this is not a valid general-block correctness argument.

Exact Practice Q5 result
Assuming an NVIDIA warp size of 32, T x T <= 32 for T=1,2,3,4,5, so the block fits in one warp and the given instruction sequence works in the intended lecture model. For T>=6, multiple warps create a race. Insert __syncthreads() after the shared store to support every T from 1 to 20. Treat BLOCK_SIZE and BLOCK_WIDTH as the same compile-time tile size in the question.

5.4 Matrix-multiplication load count

Each C[i,j] performs N multiply-add iterations and requests A[i,k] and B[k,j]. There are N^2 outputs, so the naive algorithm issues N^3 requests for A and N^3 for B. A particular A[i,k] is requested once for each j, and a particular B[k,j] once for each i.

Naive: each input element is requested N times; total input requests = 2N^3

A T x T output tile loads an A tile and B tile once, then reuses those values across T output positions. There are N/T tile stages in the k direction.

Tiled: each input element is requested N/T times; total input requests = 2N^3/T

N	Tile T	Naive requests/element	Tiled requests/element	Ideal reduction
1024	8	1024	128	8x
1024	16	1024	64	16x
1024	32	1024	32	32x

Actual DRAM transactions depend on cache hits, alignment, and write policy. The practice question asks for algorithmic requests to global memory including caches, so N and N/T are the intended counts.

5.5 Tiled GEMM skeleton

```
__global__ void mm(const float* A, const float* B, float* C, int N) {
  __shared__ float As[T][T], Bs[T][T];
  int row = blockIdx.y*T + threadIdx.y;
  int col = blockIdx.x*T + threadIdx.x;
  float sum = 0.0f;
  for (int m = 0; m < (N+T-1)/T; ++m) {
    As[ty][tx] = (row<N && m*T+tx<N) ? A[row*N+m*T+tx] : 0;
    Bs[ty][tx] = (col<N && m*T+ty<N) ? B[(m*T+ty)*N+col] : 0;
    __syncthreads();
    for (int k=0; k<T; ++k) sum += As[ty][k] * Bs[k][tx];
    __syncthreads();
  }
  if (row<N && col<N) C[row*N+col] = sum;
}
```

The first barrier ensures that tile loading is complete. The second prevents a fast warp from overwriting the shared tile for the next iteration while a slow warp is still consuming the current tile.

5.6 Resources and occupancy

Registers per thread, shared memory per block, and threads per block limit the number of blocks and warps that can reside on an SM. Larger blocks are not automatically faster. Use enough active warps to hide latency while monitoring register spills, shared-memory capacity, instruction-level parallelism, and bandwidth.

Symptom	Possible cause	Check or response
Low occupancy	Too many registers or too much shared memory	Inspect compiler resource reports; tune block size
High occupancy but slow	Transactions, bank conflicts, or divergence	Profile access and control behavior

Symptom	Possible cause	Check or response
Shared memory made it slower	Too little reuse or too much barrier/conflict cost	Compute load reduction and bank mapping
Sensitive to block size	Warp count and resource allocation granularity	Benchmark several legal candidates

6. Pthreads, OpenMP, MPI, and Hybrid Models

6.1 Model comparison

Interface	Execution unit	Data organization	Programmer responsibility
Pthreads	CPU thread	Shared address space	Create threads, divide work, synchronize
OpenMP	Thread/task	Shared host data	Mark parallel work and data attributes
CUDA/HIP	GPU thread/block	Host and device spaces	Map work and manage data movement
MPI	Process/rank	Process-local memory	Distribute data and communicate
MPI+X	Rank plus threads/GPU	Distributed across nodes, shared within a node	Manage hierarchical parallelism and affinity

6.2 OpenMP fork-join and worksharing

```
double sum = 0.0;
#pragma omp parallel for reduction(+:sum) schedule(static)
for (int i = 0; i < n; ++i)
    sum += a[i];
```

parallel creates a team, for distributes iterations, and reduction creates private partial results before combining them. A plain shared sum update would create a data race.

Construct/clause	Meaning	Key caution
parallel	Create a team for a structured block	Know the default data environment
for	Distribute loop iterations	No unsafe loop-carried dependency
sections	Distribute distinct code sections	Section count limits parallelism
single	One thread executes; implicit barrier by default	Different semantics from master
private	Uninitialized per-thread copy	Original value is not inherited
firstprivate	Per-thread copy initialized from original	Copying large objects may cost
lastprivate	Publish the logically last iteration's value	Not based on completion order
reduction	Private partial values plus combine	Floating-point order may change results slightly

6.3 Synchronization choices

Primitive	Scope	Good use	Cost/caution
barrier	Whole team	Phase boundary	Waits for the slowest thread
critical	Named critical region	Compound shared update	Serial bottleneck under contention
atomic	One supported memory update	Simple read-modify-write	Narrower than critical, still contended
lock	Explicit lock object	Complex mutual exclusion	Deadlock and lock-order risks
ordered	Logical loop order	Small order-sensitive region	Reduces parallelism
taskwait	Child tasks of the current task	Task dependency phases	Not a whole-team barrier

6.4 OpenMP memory model and tasks

Threads may observe memory through registers, caches, and other temporary views. Synchronization constructs establish required ordering and visibility. Prefer barriers, atomics, and locks over hand-built flush protocols when possible.

```
#pragma omp parallel
#pragma omp single
{
  #pragma omp task shared(x) depend(out:x)
  produce(x);
  #pragma omp task shared(x) depend(in:x)
  consume(x);
}
```

Tasks express units of work and dependencies while the runtime schedules them on threads. They are useful for recursion, linked structures, and irregular graphs. Tasks that are too small lose to creation and scheduling overhead.

6.5 MPI matrix-vector example

With row-partitioned matrix-vector multiplication, each rank owns a subset of matrix rows but needs the whole x vector. The vector can be broadcast from a root or reconstructed everywhere with all-gather if it is initially distributed. Each rank computes local y values, then gathers them if a centralized result is needed.

7. Solved Practice Midterm

7.1 Q1 True/False

Item	Answer	Reason
(a)	True	Scratchpad/shared memory is programmer-managed, so explicit load/store operations are required.
(b)	True	GPUs generally devote more resources to throughput and concurrent threads than to complex control.
(c)	True	A barrier requires all members of the participant set and is a collective synchronization primitive.
(d)	True	Gustafson describes fixed-time/scaled-problem speedup as processor count grows.
(e)	True	Roofline compares operational intensity against memory and compute roofs.
(f)	False	Performance stops increasing with I after the compute roof is reached.
(g)	False	Insufficient reuse, bank conflicts, and barrier overhead can make shared memory slower.
(h)	False	Hardware preserves correctness under divergence, but serial path execution can cost throughput.
(i)	True	cudaMallocManaged() allocates Unified Memory.
(j)	False	If all threads in a warp take the same path, the branch is uniform rather than divergent.
(k)	True*	In the course's dynamic-superscalar context, hardware selects ready instructions. Superscalar and out-of-order are not strictly identical.
(l)	True	One vector instruction describes operations on several elements and reduces fetch/decode demand.
(m)	False	Warp/wavefront width is architecture-defined; software chooses block/work-group dimensions.
(n)	True	Arm Valhall performs dynamic instruction scheduling in hardware.

7.2 Q2 Terms

(a) Simultaneous multithreading (SMT): A physical core maintains state for multiple hardware threads and can issue instructions from more than one thread into shared execution resources, filling otherwise idle pipeline slots.

(b) All-gather communication: Each process contributes a distinct piece of data, and every process receives the concatenation of all pieces.

(c) GPU texture cache: A read-only cache path optimized for spatial locality, especially 2D access patterns, with graphics-oriented addressing/filtering support where available.

7.3 Q3 Amdahl calculation

s=0.1 and the parallel fraction is 0.9. With p=8: Tp=0.1+0.9/8=0.2125 and S=1/0.2125=4.7059. With p approaching infinity: Smax=1/0.1=10.

7.4 Q4 Two-section index

Each block owns 2*blockDim.x elements and threads first process a consecutive section, so the first index is:

$$i = blockIdx.x * (2 * blockDim.x) + threadIdx.x$$

The second element is i+blockDim.x. Multiplying the ordinary global thread index by 2 produces first addresses 0,2,4,... and does not match the two-section, coalesced mapping.

7.5 Q5 Transpose correctness

The shared store is not followed by a barrier. With T=BLOCK_WIDTH and a 32-thread NVIDIA warp, T^2<=32 for T=1..5, so the intended one-warp instruction sequence works. T=6..20 creates multiple warps and a race is possible. Add __syncthreads() between the shared store and transposed load.

```
blockA[threadIdx.y][threadIdx.x] = A_elements[baseIdx];
__syncthreads();
A_elements[baseIdx] = blockA[threadIdx.x][threadIdx.y];
```

7.6 Q6 Coalesced first elements

The correct choice is (a): i=(blockIdx.x*blockDim.x)*2+threadIdx.x. As threadIdx.x runs 0,1,2,... within a warp, the first addresses are consecutive. The second section can use i+blockDim.x.

7.7 Q7 Matrix-multiplication requests

Without tiling, every A or B input element is requested N times. With T x T shared-memory tiles, every input element is requested N/T times because one global load is reused for T output positions.

7.8 Q8 Shared-variable instances

There are **1000 logical instances**, one per block over the kernel lifetime. Physical shared storage is reused across blocks that become resident at different times.

7.9 Q9 Why shared memory instead of registers?

The central reason is **cross-thread sharing and reuse**. Registers are private to one thread. Shared memory allows block threads to cooperatively load global data once, synchronize, reuse one another's values, and rearrange accesses for operations such as transpose and tiled GEMM.

8. Additional Exam-Style Questions

Question 1. A program has a 4% serial fraction. Find Amdahl speedup and efficiency on 16 processors.

Answer: S=1/[0.04+0.96/16]=10. E=10/16=0.625=62.5%.

Question 2. A device has 15 TFLOP/s peak and 750 GB/s bandwidth. Find the ridge point and the roof for I=8 FLOP/B.

Answer: Ridge=15000/750=20 FLOP/B. I=8 is memory-bound with a 6 TFLOP/s roof.

Question 3. Every thread in a warp evaluates an if-condition as false. Is there divergence?

Answer: No. The decision is uniform and only one path executes.

Question 4. A kernel stages data in shared memory but does not reduce global loads. Is it guaranteed to improve?

Answer: No. Shared loads/stores, barriers, and bank conflicts add cost without reuse.

Question 5. An OpenMP parallel loop performs sum += a[i] with shared sum. What is wrong?

Answer: It has a data race. Use reduction(+:sum), or a usually more expensive atomic/critical region.

Question 6. Which schedule better fits an irregular graph frontier: static or dynamic?

Answer: Usually dynamic, because work can be redistributed. Choose chunk size to control overhead.

Question 7. A grid has 2048 blocks but only 16 can be resident at once. How many logical shared-array instances exist?

Answer: 2048 over the kernel lifetime; physical storage is reused by resident blocks.

Question 8. A tiled GEMM has the first barrier but not the second. What race is possible?

Answer: A fast warp can overwrite the tile for the next iteration while a slow warp still reads the current tile.

Question 9. Differentiate all-gather and all-reduce.

Answer: All-gather concatenates all pieces at every rank. All-reduce combines values with an operation and returns that combined value to every rank.

Question 10. Is false sharing necessarily a correctness bug?

Answer: No. It is usually a performance bug caused by coherence traffic between independent variables in one cache line.

Five-step CUDA code check

Step	Question	Typical failure
1. Bounds	Is every global index in range?	Out-of-bounds access in the final partial block
2. Ownership	Can multiple threads conflict on a write?	Non-atomic write race
3. Dependency	Is synchronization present before reading another thread's value?	Shared producer-consumer race
4. Mapping	Does thread-to-data mapping match intent and coalescing?	Confusing per-thread and per-warp contiguity
5. Resources	Are block size, shared memory, and registers legal?	Launch failure or low occupancy

9. Final Cram Sheets

9.1 Formulas to know

Concept	Formula	Meaning
Speedup	S=T1/Tp	Improvement over serial execution
Efficiency	E=S/p	Utilization per processing element
Amdahl	S=1/[s+(1-s)/p]	Fixed size; ceiling 1/s
Gustafson	S=p-s(p-1)	Fixed time; scaled problem
Roofline	P<=min(Ppeak, B I)	Ridge at I=Ppeak/B
Communication	T=alpha+beta n	Latency for small messages, bandwidth for large
1D index	i=blockIdx.x*blockDim.x+threadIdx.x	Block offset plus local offset
Two sections	i=blockIdx.x*2B+threadIdx.x	Second element is i+B
2D row-major	idx=row*width+col	x maps to column, y maps to row
GEMM loads	N -> N/T per input	Tiling creates T-fold reuse

9.2 Instant true/false

Statement	Answer	Keyword
A barrier is collective	T	All participants arrive
Shared memory always helps	F	Reuse vs. overhead/conflicts
Hardware handles divergence without penalty	F	Path serialization
A warp taking one common path is divergent	F	Uniform control
Vector instructions reduce fetch demand	T	One instruction, many lanes
Software chooses warp size	F	Architecture-defined
cudaMallocManaged allocates Unified Memory	T	Managed allocation
Higher intensity always raises Roofline performance	F	Compute-roof saturation
Valhall uses hardware dynamic scheduling	T	Warp selection and superscalar issue

9.3 Memory-choice checklist

Need	First candidate	Why
Thread-private scalar temporary	register	Low latency and private
Share data across block threads	shared memory	Cooperation and reuse
Read-only 2D spatial locality	texture path	Spatially optimized cache path
Same read-only value for many threads	constant/broadcast path	Efficient broadcast
Large device-resident array	global memory	Capacity; requires coalescing
One convenient CPU/GPU pointer	Unified Memory	Check migration and prefetch behavior

9.4 Last self-check

- Can you state what Amdahl and Gustafson hold fixed?
- Can you derive the Roofline bound and identify the ridge point?
- Can you distinguish grid, block, warp, thread, and SM scope?
- Can you explain why divergence and coalescing are separate concerns?
- Can you explain registers vs. shared memory in terms of cross-thread sharing?
- Can you find both barriers needed by an iterative shared-memory tile?
- Can you list warp addresses when each thread handles two elements?
- Can you derive N and N/T input requests for naive and tiled GEMM?
- Can you distinguish private, firstprivate, lastprivate, and reduction?
- Can you distinguish all-gather, all-reduce, and barrier outputs?

10. References and Further Reading

The lecture materials define the exam scope. The primary documentation and original papers below were used to cross-check definitions and current API semantics. This guide paraphrases and derives rather than reproducing source text.

ID	Source	URL	Used for
[R1]	Course repository	https://github.com/k1seul/computing-2-parallel-computing	Lecture scope and practice midterm
[R2]	NVIDIA CUDA Programming Guide 13.2	https://docs.nvidia.com/cuda/cuda-programming-guide/	SIMT, shared memory, coalescing, Unified Memory
[R3]	CUDA C++ Best Practices Guide	https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/	Memory hierarchy and optimization
[R4]	OpenMP API Specification 6.0	https://www.openmp.org/specifications/	Directives, data environment, memory model
[R5]	MPI 4.1 Standard	https://www.mpi-forum.org/docs/mpi-4.1/	Collectives and Allgather
[R6]	Arm Valhall architecture	https://developer.arm.com/Architectures/Valhall	Dynamic scheduling and shader core
[R7]	AMD GCN3 ISA Reference	https://gpuopen.com/download/AMD_GCN3_Instruction_Set_Architecture_rev1.1.pdf	GCN execution and ISA
[R8]	Amdahl, 1967	https://doi.org/10.1145/1465482.1465560	Fixed-size speedup limit
[R9]	Gustafson, 1988	https://doi.org/10.1145/42411.42415	Scaled speedup
[R10]	Williams et al., Roofline	https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html	Operational intensity and performance ceilings

Architecture-dependent values

Warp/wavefront width, bank count, maximum threads per block, and memory-transaction details can change across architecture generations. This guide uses warp=32 for the NVIDIA CUDA context of the practice midterm and separately labels the historical AMD GCN wavefront size of 64. For a device-specific question, prioritize the stated assumptions and that generation's programming guide.

Final rule
Start calculations from the model, code questions from correctness, and performance questions from the bottleneck. When a statement says 'always,' test resource limits, access patterns, synchronization requirements, and roof saturation for a counterexample.